

FORFRESH MANUAL · 03 / 04

给新人的 Python 代码 Caveman 诠释手 册

把对象、函数、Markdown、JSONL 与错误翻成白话。目标不是背语法，而是能沿一条真实数据流找到该改的那一处。

KEIKEU CONTRIBUTORS

2026-08-26 · ROAD V0.7 同步

适用：PYTHON $\geq 3.11, < 3.14$

[下载 PDF](#)

先记住这一句

Caveman engineering 不是“随便写”，而是让数据、函数、文件和错误都能被人一步一步追踪。keikeu 的 Python 层因此优先使用普通对象、普通函数、标准库和明确边界。

- 01 keikeu_core 管模型、Markdown、Vault 与 Index； keikeu_bridge 管 DTO、service、protocol 与 Sidecar。
- 02 Markdown 是持久权威，JSON Index 是可重建投影；界面和 Rust 都不能绕过 Python 直接写作者文件。
- 03 先用 rg 找调用者，再做最小修改并跑聚焦测试；真实 Vault 永远不是练习场。

阅读路线

- | | | | |
|----|------------------|----|---------------------|
| 01 | “原始人式”到底是什么 | 02 | 先看两层地图 |
| 03 | 看懂常见 Python 句子 | 04 | 对象：数据先有形状 |
| 05 | 文件：Markdown 才是权威 | 06 | Bridge：一行 JSON 走完全程 |
| 07 | 第一次安全追代码 | 08 | 陷阱、检查与下一步 |

“原始人式” 到底是什么

尽量让一件事就是一个普通文件、一个普通对象或一个普通函数。复杂度只留在确实保护作者数据的地方，不为“以后也许会用”搭框架。

一块石头只做一件事

当前 `pyproject.toml` 把 Python 限定在 `>=3.11,<3.14`，运行时依赖列表为空；开发依赖只有 `pytest`。换句话说：能由 `dataclasses`、`pathlib`、`json` 与普通函数完成的事情，就不先加数据库、ORM、依赖注入容器或 Web 框架。这种简单不是“所有代码塞进一个文件”。真正的原则是：一个模块有一个可复述的责任，数据从入口到落盘没有暗门。新人能用 `rg` 找到名字，打开几个调用者，就应当能回答“谁验证、谁写盘、谁返回错误”。

CAVEMAN 的边界

可以朴素：普通函数、明确分支、重复两三行可读代码。不能省：信任边界校验、原子写入、并发保护、错误语义、作者内容保护。

为什么文件安全代码看起来不“简单”

作者内容不是缓存。路径 `containment`、禁止跟随不可信链接、严格 UTF-8、`schema` 校验、编辑令牌和原子替换，都是防止写错文件或覆盖并发编辑的保险。如果把这些删掉，代码行数会少，但系统会变得不可修复；那不是 `caveman`，只是失去护栏。

因此评审 Python 改动时先问两件事：它是否把产品规则放在唯一权威层？它是否保留了现有数据防护？如果答案是“是”，再讨论能否把控制流写得更直白。

先看两层地图

`keikeu_core` 是本地产品与文件规则，`keikeu_bridge` 是桌面传输与编排。先选对层，往往就省掉一半调试。

位置	白话职责	不该做什么
<code>models.py</code>	PaperV4/CardPageV4 的数据形状与值级验证	不碰 GUI、JSONL 或系统对话框

位置	白话职责	不该做什么
<code>markdown_io.py</code>	Markdown bytes、解析、渲染、CAS 与原子替换	不决定页面怎么提示用户
<code>vault.py</code>	Vault 布局、路径包含性与安全文件操作	不信任界面传来的路径
<code>indexer.py</code>	从 Markdown 重建可丢弃的 JSON Index	不让 Index 反过来覆盖 Markdown
<code>service.py</code>	把一个用例编排成若干 core 调用	不复制 parser 或文件安全规则
<code>protocol.py</code>	校验 JSONL envelope、method、params，并映射稳定错误	不把 stdout 当日志
<code>sidecar.py</code>	stdin 读一行，stdout 写一行	不持有产品状态

一条最短心智模型

- 01
- 协议把外部 JSON 变成可信 DTO，坏输入在边界被拒绝。
- 02
- service 把“保存 Paper”翻成几个明确的 core 操作。
- 03
- core 把对象渲染为 Markdown，并在安全路径上原子提交。
- 04
- Index 从已提交的 Markdown 重建；响应再沿原路返回。

如果你正在改按钮文案，Python 多半不是该去的地方；如果你正在改变 Paper 的合法形状或写盘规则，Vue 和 Rust 都不是权威。先决定“这是谁的责任”，再搜索名字。

CHAPTER 03

看懂常见 Python 句子

类型标注是路牌，dataclass 是带名字的数据盒，异常是明确拒绝。先读懂这些句型，就能覆盖项目大多数核心代码。

句型	原始人翻译
<code>code: str</code>	Paper 代号应当是字符串；类型标注不会自动清洗值。
<code>display_name: str None</code>	整份 Paper 的名称可以是字符串，也可以明确没有。
<code>pages: list[CardPageV4]</code>	这是按顺序放 CardPageV4 的列表，也是当前持久字段名。
<code>field(default_factory=list)</code>	每个新对象拿自己的空列表，不能共享同一只篮子。
<code>-> PaperV4</code>	函数承诺正常返回一个 PaperV4。
<code>raise ValueError(...)</code>	输入不合法，立刻停下并给调用者一个可解释的失败。
<code>with path.open(...) as handle</code>	进入块时打开文件，离开时一定关闭。
<code>if value is None</code>	检查“明确没有”，不要把空字符串、0 和 False 混成同一件事。

读函数，先只读签名

```
def validate_paper_code(code: str) -> str:
    if not isinstance(code, str):
        raise ValueError("code must use the K-YYYYMMDD-NNN format")
    match = _PAPER_CODE_RE.fullmatch(code)
    if match is None:
        raise ValueError("code must use the K-YYYYMMDD-NNN format")
    try:
        datetime.strptime(match.group(1), "%Y%m%d")
    except ValueError:
        raise ValueError("code must contain a valid calendar date") from None
    if not 1 <= int(match.group(2)) <= 999:
        raise ValueError("code sequence must be between 001 and 999")
    return code
```

先看输入和输出：拿一个字符串，正常时原样还回 canonical code。真实函数不做 `strip()`，而是要求精确的 `K-YYYYMMDD-NNN`、合法日历日期和 `001-999` 序号；调用者不能靠它静默修剪输入。

`try/except` 不是橡皮擦

只在这一层能做有用事情时捕获异常：补充路径上下文、把底层异常映射成稳定协议错误，或执行必要清理。`except Exception: pass` 会把损坏或数据丢失信号吞掉；宽捕获若没有明确恢复动作，通常是错误。

对象：数据先有形状

模型让一篇 Paper 在进入文件层之前就有可检查的字段与不变量。它不是数据库模型，也不负责把自己保存到磁盘。

@dataclass 是什么

在 `models.py` 里，`CardPageV4` 与 `PaperV4` 使用 `dataclass`。装饰器替我们生成初始化和比较等机械代码，字段本身仍是普通 Python 值。这比手写几十行赋值更短，也没有引入外部框架。

`__post_init__` 在 `dataclass` 初始化完成后运行。项目用它执行字段规范化与值级验证，所以“对象已经创建”不代表输入可以任意；无效值会尽早失败，而不是等到写盘后才暴露。

重要区别

类型标注说“预期是什么”，运行时守卫说“实际允许什么”。二者缺一不可：`str` 不会自动阻止空字符串，`list[str]` 也不会替你检查业务顺序。

不要给模型塞进不属于它的能力

模型可以知道自己的字段是否合法，却不该弹原生对话框、打开 Finder、调用 Tauri 或写用户界面状态。文件位置和持久化条件交给 Vault/Markdown 层；用例顺序交给 service。这样一个模型可在纯单元测试里构造，不需要桌面环境。

增加字段前要同时检查 Markdown schema、解析/渲染、DTO、协议与测试，但不能为了“也许以后扩展”先做通用属性袋。未知 frontmatter 需要按规则保留时，采用已有机制，而不是把所有字段变成无法验证的字典。

文件：Markdown 才是权威

作者可读的 Markdown 是持久真相，Index JSON 只是方便查询的投影。读写必须可验证、可恢复，并且不能静默改变作者内容。

读一份 Paper

01 从已验证的 Vault 相对路径取得文件，不接受任意绝对路径。

02 读取 bytes，严格按 UTF-8 解码；坏编码是错误，不用替换字符糊过去。

- 03解析 frontmatter 和正文，检查 schema 与字段形状。
- 04构造 `PaperV4`，让模型守卫再次检查值级不变量。
- 05把内容与编辑令牌交给调用者；令牌用于防止盲目覆盖外部修改。

写一份 Paper

写入不是“`path.write_text(...)` 就结束”。项目先渲染确定性的 Markdown bytes，再检查目标仍在 Vault 内、旧编辑令牌仍匹配，最后使用原子替换提交。这样进程中途失败时，不会把半份文件留在作者目录里。

若同一文件已被外部编辑器改动，CAS 检查应拒绝旧写入，而不是悄悄覆盖。刻意留空的可选字段也不能被“看起来更整洁”的规范化吞掉；未知 frontmatter 在可行时应保留。

Index 是影子，不是身体

`indexer.py` 从 Markdown 扫描并生成 JSON Index。Index 损坏或丢失时，正确动作是重建；不能拿旧 Index 回写 Markdown。记住这条单向箭头：



练习安全

涉及迁移、删除、恢复或持久配置时，先在 fixture 或复制品上操作。不要用唯一真实 Vault 验证“应该没事”。

CHAPTER 06

Bridge：一行 JSON 走完全程

Sidecar 不是 HTTP 服务器。它从 stdin 接收一行 JSONL，经 protocol 与 service 进入 core，再从 stdout 返回恰好一行结构化响应。

从 Tauri 到 Python

- 01Rust 写入带 `v`、`id`、`session`、`method` 与 `params` 的一行 JSON。
- 02`sidecar.py` 从 stdin 读取，不自行解释业务。
- 03`protocol.py` 严格检查重复字段、非有限数字、版本、会话、方法与参数。

04 `service.py` 把 DTO 转成 core 操作。

05 protocol 形成 `ok/result/error` envelope, Sidecar flush 到 stdout。

```
{"v":2,"id":41,"session":"...","method":"library.query","params":{"...}}
{"v":2,"id":41,"ok":true,"result":{"...}}
```

stdout 是协议线，不是留言板

普通 `print()` 默认写 stdout。Sidecar 环境里，这会被 Rust 当成 JSONL 响应，导致协议不匹配并阻塞运行时。必要诊断只能写 stderr，而且不能泄漏作者正文、私有路径、token 或未发布草稿。

为什么 service 仍然需要一层

protocol 只懂“外部请求是否合法”，core 只懂“模型和文件规则是什么”；service 负责一个用例的步骤顺序。例如保存后重建 Index、把核心错误翻成 DTO。若把这一编排塞进 parser 或模型，传输与产品规则就会缠在一起。

CHAPTER 07

第一次安全追代码

不要从“看起来相关”的文件开始改。先搜索入口和所有调用者，再沿真实流向缩小到唯一责任点。

例：你要理解 `paper.save`

```
rg -n 'paper\.save|save_paper' src tests frontend/src frontend/src-tauri/src
rg -n 'def .*save|class .*Paper' src/keikeu_core src/keikeu_bridge
rg -n 'save' tests/test_markdown_io.py tests/test_bridge_protocol.py
```

第一条找跨语言名字，第二条找 Python 实现，第三条找现有证据。读 protocol 的 dispatcher、service 方法、core 写入函数和它们的测试；不要只读报错位置。一个共享函数若是根因，在那里修一次通常比给每个调用者打补丁更小。

改动前写出三句话

- 输入：谁把什么数据交给这层？哪些值还不可信？
- 不变量：哪些规则若被破坏会覆盖内容、逃出 Vault 或丢失恢复语义？
- 输出：正常结果和失败如何被上一层看见？

最小证据

```
.venv/bin/python -m pytest tests/test_models.py
.venv/bin/python -m pytest tests/test_markdown_io.py
.venv/bin/python -m pytest tests/test_bridge_protocol.py
.venv/bin/python -m compileall -q src
```

只改一个纯函数时先跑直接测试，再按风险扩到完整 pytest。`compileall` 只证明文件可编译，不证明业务正确；Sidecar 构建只证明可打包，不证明桌面产品已验收。证据名字要说准确。

CHAPTER 08

陷阱、检查与下一步

最常见的问题不是 Python 语法，而是越层、吞错、绕过文件守卫或把派生数据当权威。守住边界后，改动通常很小。

七条禁止条件反射

1. 不要让 `keikeu_core` import Vue、Tauri、Flet 或任何 GUI/transport。
2. 不要在 GUI 或 Rust 里直接写 Markdown、Index JSON 或迁移结果。
3. 不要把 Index 当数据库，也不要 Index 覆盖作者 Markdown。
4. 不要对未知输入做宽泛 `except Exception` 后继续运行。
5. 不要用 `Path.resolve()` 一句话取代项目已有的 no-follow 与 `containment` 守卫。
6. 不要静默修剪、重排或补写作者文字；规范化必须由产品规则明确授权。
7. 不要在真实唯一 Vault 上练迁移、删除、恢复或并发冲突。

一次小改动的顺序

- | | |
|----|---|
| 01 | 读 SPEC、RULES 与相关源码，确认责任层。 |
| 02 | 运行 Git gate，声明要改与不改的文件，搜索所有调用者。 |
| 03 | 只改根因处，并补一个能在回归时失败的聚焦测试。 |
| 04 | 先跑直接 pytest，再按风险运行完整套件、Sidecar 构建或 Tauri smoke。 |
| 05 | 检查 diff 与工作树，准确报告已运行和未运行的证据。 |

权威边界：这本手册帮助读代码，不定义产品或工程规则。发生冲突时，以 SPEC、RULES、PROJECT 与当前 `src/`、`tests/` 运行事实为准；修手册，不复制一套新规则。

继续查证

1. 本项目 Python 版本、依赖与 pytest 配置
2. 当前模型实现、Markdown I/O、Vault 安全边界
3. 当前 JSONL protocol、service 编排、Sidecar 入口
4. Python 官方教程 与 错误与异常
5. dataclasses 官方文档 与 pathlib 官方文档